

```
In [1]: |pip install pandas openpyxl numpy matplotlib seaborn scikit-learn

Requirement already satisfied: pandas in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (3.0.5)
Requirement already satisfied: openpyxl in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (3.1.5)
Requirement already satisfied: numpy in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (2.5.1)
Requirement already satisfied: matplotlib in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (3.11.1)
Requirement already satisfied: seaborn in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (1.9.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2026.3)
Requirement already satisfied: et_xmlfile in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: nanmwals>=2.0.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.24.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 26.2 -> 26.2.1
[notice] To update, run: C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

print("Libraries imported successfully!")

Libraries imported successfully!

In [3]: |pip install --upgrade matplotlib seaborn numexpr bottleneck

Requirement already satisfied: matplotlib in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (3.11.1)
Requirement already satisfied: seaborn in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (0.13.2)
Collecting numexpr
  Downloading numexpr-2.14.2-cp314-cp314-win_amd64.whl.metadata (9.2 kB)
Collecting bottleneck
  Downloading bottleneck-1.6.0-cp314-cp314-win_amd64.whl.metadata (8.4 kB)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (2.5.1)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from seaborn) (3.0.5)
Requirement already satisfied: tzdata in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas>=1.2->seaborn) (2026.3)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading numexpr-2.14.2-cp314-cp314-win_amd64.whl (160 kB)
Downloading bottleneck-1.6.0-cp314-cp314-win_amd64.whl (115 kB)
Installing collected packages: numexpr, bottleneck

----- 0/2 [numexpr]
----- 1/2 [bottleneck]
----- 2/2 [bottleneck]

Successfully installed bottleneck-1.6.0 numexpr-2.14.2
[notice] A new release of pip is available: 26.2 -> 26.2.1
[notice] To update, run: C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

print("All libraries imported successfully!")

All libraries imported successfully!

In [7]: import pandas as pd
import os

file_path = r"C:\Users\tvija\Downloads\sales_data.csv"

print(os.path.exists(file_path))

True

In [8]: import pandas as pd

file_path = r"C:\Users\tvija\Downloads\sales_data.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print(df.head())

Dataset loaded successfully!
Shape: (1000, 14)
  Product_ID  Sale_Date  Sales_Rep  Region  Sales_Amount  Quantity_Sold  \
0         1052    2/3/2023      Bob   North      5053.97             18
1         1093    4/21/2023      Bob    West      4384.02             17
2         1015    9/21/2023    David   South      4631.23             30
3         1072    8/24/2023      Bob   South      2167.94             39
4         1061    3/24/2023    Charlie  East      3750.20             13

  Product_Category  Unit_Cost  Unit_Price  Customer_Type  Discount  \
0      Furniture      152.75      267.22      Returning      0.09
1      Furniture     3816.39     4209.44      Returning      0.11
2         Food       261.56       371.40      Returning      0.20
3      Clothing     4330.03     4467.75         New         0.02
4      Electronics     637.37     692.71         New         0.08

  Payment_Method  Sales_Channel  Region_and_Sales_Rep
0          Cash          Online          North-Bob
1          Cash          Retail          West-Bob
2  Bank Transfer          Retail        South-David
3   Credit Card          Retail          South-Bob
4   Credit Card          Online          East-Charlie

In [9]: df.head()

Out[9]:   Product_ID  Sale_Date  Sales_Rep  Region  Sales_Amount  Quantity_Sold  Product_Category  Unit_Cost  Unit_Price  Customer_Type  Discount  Payment_Method  Sales_Channel  Region_and_Sales_Rep
0         1052    2/3/2023      Bob   North      5053.97             18      Furniture     152.75      267.22      Returning      0.09          Cash          Online          North-Bob
1         1093    4/21/2023      Bob    West      4384.02             17      Furniture     3816.39     4209.44      Returning      0.11          Cash          Retail          West-Bob
2         1015    9/21/2023    David   South      4631.23             30         Food       261.56       371.40      Returning      0.20  Bank Transfer          Retail        South-David
3         1072    8/24/2023      Bob   South      2167.94             39      Clothing     4330.03     4467.75         New         0.02   Credit Card          Retail          South-Bob
4         1061    3/24/2023    Charlie  East      3750.20             13      Electronics     637.37     692.71         New         0.08   Credit Card          Online          East-Charlie

In [10]: df.tail()
```

Out[10]:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
995	1010	4/15/2023	Charlie	North	4733.88	4	Food	4943.03	5442.15	Returning	0.29	Cash	Online	North-Charlie
996	1067	9/7/2023	Bob	North	4716.36	37	Clothing	1754.32	1856.40	New	0.21	Bank Transfer	Retail	North-Bob
997	1018	4/27/2023	David	South	7629.70	17	Clothing	355.72	438.27	Returning	0.06	Bank Transfer	Online	South-David
998	1100	12/20/2023	David	West	1629.47	39	Electronics	3685.03	3743.39	New	0.01	Bank Transfer	Online	West-David
999	1086	8/16/2023	Alice	East	4923.93	48	Food	2632.58	2926.68	Returning	0.14	Cash	Online	East-Alice

In [11]:

df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
Column Non-Null Count Dtype
--- ---
0 Product_ID 1000 non-null int64
1 Sale_Date 1000 non-null str
2 Sales_Rep 1000 non-null str
3 Region 1000 non-null str
4 Sales_Amount 1000 non-null float64
5 Quantity_Sold 1000 non-null int64
6 Product_Category 1000 non-null str
7 Unit_Cost 1000 non-null float64
8 Unit_Price 1000 non-null float64
9 Customer_Type 1000 non-null str
10 Discount 1000 non-null float64
11 Payment_Method 1000 non-null str
12 Sales_Channel 1000 non-null str
13 Region_and_Sales_Rep 1000 non-null str
dtypes: float64(4), int64(2), str(8)
memory usage: 109.5 KB

In [12]:

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

Number of rows: 1000
Number of columns: 14

In [13]:

print("Column names:")
print(df.columns.tolist())

Column names:
['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

In [14]:

print("Data Types:")
print(df.dtypes)

Data Types:
Product_ID int64
Sale_Date str
Sales_Rep str
Region str
Sales_Amount float64
Quantity_Sold int64
Product_Category str
Unit_Cost float64
Unit_Price float64
Customer_Type str
Discount float64
Payment_Method str
Sales_Channel str
Region_and_Sales_Rep str
dtype: object

In [16]:

import os

raw_folder = r"C:\Users\tvija\Downloads\salary_mlops\data\raw"

os.makedirs(raw_folder, exist_ok=True)

raw_path = os.path.join(raw_folder, "salary_data.xlsx")

print("Raw folder:", raw_folder)
print("Raw file path:", raw_path)

Raw folder: C:\Users\tvija\Downloads\salary_mlops\data\raw
Raw file path: C:\Users\tvija\Downloads\salary_mlops\data\raw\salary_data.xlsx

In [17]:

print("Missing values:")
print(df.isnull().sum())

Missing values:
Product_ID 0
Sale_Date 0
Sales_Rep 0
Region 0
Sales_Amount 0
Quantity_Sold 0
Product_Category 0
Unit_Cost 0
Unit_Price 0
Customer_Type 0
Discount 0
Payment_Method 0
Sales_Channel 0
Region_and_Sales_Rep 0
dtype: int64

In [18]:

df.describe()

Out[18]:

	Product_ID	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price	Discount
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	1050.128000	5019.265230	25.355000	2475.304550	2728.440120	0.15239
std	29.573505	2846.790126	14.159006	1417.872546	1419.39839	0.08720
min	1001.000000	100.120000	1.000000	60.280000	167.120000	0.00000
25%	1024.000000	2550.297500	13.000000	1238.380000	1509.085000	0.08000
50%	1051.000000	5019.300000	25.000000	2467.235000	2696.400000	0.15000
75%	1075.000000	7507.445000	38.000000	3702.865000	3957.970000	0.23000
max	1100.000000	9989.040000	49.000000	4995.300000	5442.150000	0.30000

In [20]:

print(df.columns.tolist())

['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

In [21]:

Check missing values
print("Missing values:")
print(df.isnull().sum())

Check duplicates
print("\nDuplicate rows:", df.duplicated().sum())

Basic statistics

```
print("\nStatistical summary:")
print(df.describe())
```

Missing values:

Product_ID	0
Sale_Date	0
Sales_Rep	0
Region	0
Sales_Amount	0
Quantity_Sold	0
Product_Category	0
Unit_Cost	0
Unit_Price	0
Customer_Type	0
Discount	0
Payment_Method	0
Sales_Channel	0
Region_and_Sales_Rep	0

dtype: int64

Duplicate rows: 0

Statistical summary:

	Product_ID	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	1050.128000	5019.265230	25.355000	2475.304550	2728.440120
std	29.573505	2846.790126	14.159006	1417.872546	1419.399839
min	1001.000000	100.120000	1.000000	60.280000	167.120000
25%	1024.000000	2550.297500	13.000000	1238.380000	1509.085000
50%	1051.000000	5019.300000	25.000000	2467.235000	2696.400000
75%	1075.000000	7507.445000	38.000000	3702.865000	3957.970000
max	1100.000000	9989.040000	49.000000	4995.300000	5442.150000

	Discount
count	1000.000000
mean	0.15239
std	0.08720
min	0.000000
25%	0.000000
50%	0.150000
75%	0.230000
max	0.300000

```
In [23]: # Check missing values
print("Missing values:")
print(df.isnull().sum())

# Check duplicates
print("\nDuplicate rows:", df.duplicated().sum())

# Dataset information
print("\nDataset information:")
df.info()
```

Missing values:

Product_ID	0
Sale_Date	0
Sales_Rep	0
Region	0
Sales_Amount	0
Quantity_Sold	0
Product_Category	0
Unit_Cost	0
Unit_Price	0
Customer_Type	0
Discount	0
Payment_Method	0
Sales_Channel	0
Region_and_Sales_Rep	0

dtype: int64

Duplicate rows: 0

Dataset information:

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product_ID            1000 non-null  int64
1   Sale_Date             1000 non-null  str
2   Sales_Rep             1000 non-null  str
3   Region                1000 non-null  str
4   Sales_Amount          1000 non-null  float64
5   Quantity_Sold         1000 non-null  int64
6   Product_Category      1000 non-null  str
7   Unit_Cost              1000 non-null  float64
8   Unit_Price            1000 non-null  float64
9   Customer_Type         1000 non-null  str
10  Discount              1000 non-null  float64
11  Payment_Method        1000 non-null  str
12  Sales_Channel         1000 non-null  str
13  Region_and_Sales_Rep  1000 non-null  str
dtypes: float64(4), int64(2), str(8)
memory usage: 109.5 KB
```

```
In [24]: print("Number of duplicate rows:")
print(df.duplicated().sum())

Number of duplicate rows:
0
```

```
In [25]: before = len(df)

df = df.drop_duplicates()

after = len(df)

print("Duplicate rows removed:", before - after)
print("New shape:", df.shape)

Duplicate rows removed: 0
New shape: (1000, 14)
```

```
In [26]: numerical_columns = [
        "Age",
        "Years of Experience"
    ]

print("Numerical columns:")
print(numerical_columns)

Numerical columns:
['Age', 'Years of Experience']
```

```
In [27]: categorical_columns = [
        "Gender",
        "Education Level",
        "Job Title"
    ]

print("Categorical columns:")
print(categorical_columns)

Categorical columns:
['Gender', 'Education Level', 'Job Title']
```

```
In [28]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numerical_columns
        ),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ]
)

print("Preprocessing pipeline created successfully!")
```

Preprocessing pipeline created successfully!

```
In [29]: from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (
    RandomForestRegressor,
    GradientBoostingRegressor
)

print("Models imported successfully!")
```

Models imported successfully!

```
In [30]: models = {

    "Linear Regression":
        LinearRegression(),

    "Decision Tree":
        DecisionTreeRegressor(
            random_state=42,
            max_depth=10
        ),

    "Random Forest":
        RandomForestRegressor(
            n_estimators=100,
            random_state=42
        ),

    "Gradient Boosting":
        GradientBoostingRegressor(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=3,
            random_state=42
        )
}

print("Models created successfully!")

for model in models:
    print(model)
```

Models created successfully!
 Linear Regression
 Decision Tree
 Random Forest
 Gradient Boosting

```
In [33]: # =====
# SALARY PREDICTION - COMPLETE MACHINE LEARNING CODE
# =====

# -----
# 1. IMPORT LIBRARIES
# -----

import os
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (
    RandomForestRegressor,
    GradientBoostingRegressor
)

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# -----
# 2. LOAD DATA
# -----

file_path = r"C:\Users\tvija\Downloads\Salary Data.xlsx"

# Check whether file exists
if not os.path.exists(file_path):
    print("ERROR: Salary Data.xlsx was not found!")
    print("\nPython searched here:")
    print(file_path)
    print("\nExcel files currently in Downloads:")

    downloads_folder = r"C:\Users\tvija\Downloads"

    for file in os.listdir(downloads_folder):
        if file.lower().endswith((".xlsx", ".xls")):
            print(file)
else:
    df = pd.read_excel(file_path)

    print("Dataset loaded successfully!")
    print("Original shape:", df.shape)

# -----
# 3. CLEAN COLUMN NAMES
# -----

df.columns = df.columns.str.strip()

print("\nColumn names:")
```

```

print(df.columns.tolist())

# -----
# 4. CHECK REQUIRED COLUMNS
# -----

required_columns = [
    "Age",
    "Gender",
    "Education Level",
    "Job Title",
    "Years of Experience",
    "Salary"
]

missing_columns = [
    col for col in required_columns
    if col not in df.columns
]

if missing_columns:

    print("\nERROR: Required columns are missing:")
    print(missing_columns)

    print("\nYour actual columns are:")
    print(df.columns.tolist())

else:

    # -----
    # 5. REMOVE DUPLICATES
    # -----

    before_duplicates = df.shape[0]

    df = df.drop_duplicates()

    after_duplicates = df.shape[0]

    print("\nDuplicates removed:",
          before_duplicates - after_duplicates)

    print("Shape after removing duplicates:", df.shape)

    # -----
    # 6. CHECK MISSING VALUES BEFORE CLEANING
    # -----

    print("\nMissing values before cleaning:")

    print(df.isnull().sum())

    # -----
    # 7. HANDLE MISSING VALUES
    # -----

    # Numerical columns

    df["Age"] = df["Age"].fillna(
        df["Age"].median()
    )

    df["Years of Experience"] = df[
        "Years of Experience"
    ].fillna(
        df["Years of Experience"].median()
    )

    df["Salary"] = df["Salary"].fillna(
        df["Salary"].median()
    )

    # Categorical columns

    df["Gender"] = df["Gender"].fillna(
        df["Gender"].mode()[0]
    )

    df["Education Level"] = df[
        "Education Level"
    ].fillna(
        df["Education Level"].mode()[0]
    )

    df["Job Title"] = df[
        "Job Title"
    ].fillna(
        df["Job Title"].mode()[0]
    )

    print("\nMissing values handled successfully!")

    # -----
    # 8. CHECK MISSING VALUES AFTER CLEANING
    # -----

    print("\nMissing values after cleaning:")

    print(df.isnull().sum())

    # -----
    # 9. FEATURE ENGINEERING
    # -----

    # Experience-to-age ratio

    df["Experience_Ratio"] = (
        df["Years of Experience"] /
        df["Age"].replace(0, np.nan)
    )

    df["Experience_Ratio"] = (
        df["Experience_Ratio"].fillna(0)
    )

    # Age groups

    df["Age_Group"] = pd.cut(
        df["Age"],
        bins=[0, 25, 35, 45, 60, 100],
        labels=[
            "Young",
            "Early Career",
            "Mid Career",
            "Senior",
        ]
    )

```

```

        "Late Career"
    ]
)

print("\nFeature engineering completed!")

print("\nNew features:")
print([
    "Experience_Ratio",
    "Age_Group"
])

# -----
# 10. FEATURES AND TARGET
# -----

X = df.drop(
    "Salary",
    axis=1
)

y = df["Salary"]

print("\nFeatures:")
print(X.columns.tolist())

print("\nTarget:")
print("Salary")

# -----
# 11. TRAIN TEST SPLIT
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining data:", X_train.shape)
print("\nTesting data:", X_test.shape)

# -----
# 12. DEFINE NUMERICAL FEATURES
# -----

numeric_features = [
    "Age",
    "Years of Experience",
    "Experience_Ratio"
]

# -----
# 13. DEFINE CATEGORICAL FEATURES
# -----

categorical_features = [
    "Gender",
    "Education_Level",
    "Job Title",
    "Age_Group"
]

# -----
# 14. PREPROCESSOR
# -----

preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            StandardScaler(),
            numeric_features
        ),
        (
            "categorical",
            OneHotEncoder(
                handle_unknown="ignore"
            ),
            categorical_features
        )
    ]
)

# -----
# 15. DEFINE MODELS
# -----

models = {
    "Linear Regression":
        LinearRegression(),

    "Decision Tree":
        DecisionTreeRegressor(
            random_state=42,
            max_depth=10
        ),

    "Random Forest":
        RandomForestRegressor(
            n_estimators=100,
            random_state=42
        ),

    "Gradient Boosting":
        GradientBoostingRegressor(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=3,
            random_state=42
        )
}

# -----
# 16. TRAIN MODELS
# -----

trained_models = []

results = []

```

```

for name, model in models.items():

    print("\n" + "=" * 50)
    print("Training:", name)
    print("=" * 50)

    pipeline = Pipeline(
        steps=[
            (
                "preprocessor",
                preprocessor
            ),
            (
                "model",
                model
            )
        ]
    )

    # Train model

    pipeline.fit(
        X_train,
        y_train
    )

    # Save trained model

    trained_models.append(
        (name, pipeline)
    )

    # Predictions

    predictions = pipeline.predict(
        X_test
    )

    # -----
    # Evaluation metrics
    # -----

    mae = mean_absolute_error(
        y_test,
        predictions
    )

    mse = mean_squared_error(
        y_test,
        predictions
    )

    rmse = np.sqrt(mse)

    r2 = r2_score(
        y_test,
        predictions
    )

    # Save results

    results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2
    })

    # Display results

    print("MAE :", round(mae, 2))

    print("MSE :", round(mse, 2))

    print("RMSE:", round(rmse, 2))

    print("R²  :", round(r2, 4))

    # -----
    # 17. MODEL COMPARISON
    # -----

    results_df = pd.DataFrame(
        results
    )

    results_df = results_df.sort_values(
        by="R2",
        ascending=False
    ).reset_index(
        drop=True
    )

    print("\n")
    print("=" * 60)
    print("MODEL COMPARISON")
    print("=" * 60)

    print(results_df)

    # -----
    # 18. BEST MODEL
    # -----

    best_model_name = results_df.loc[
        0,
        "Model"
    ]

```

```
best_model = dict(
    trained_models
)[best_model_name]

best_r2 = results_df.loc[
    0,
    "R2"
]

print("\n")
print("=" * 60)
print("BEST MODEL")
print("=" * 60)

print("Model:", best_model_name)

print(
    "R² Score:",
    round(best_r2, 4)
)

# -----
# 19. SAMPLE PREDICTIONS
# -----

best_predictions = best_model.predict(
    X_test
)

comparison = pd.DataFrame({
    "Actual Salary":
        y_test.values,

    "Predicted Salary":
        best_predictions
})

comparison["Difference"] = (
    comparison["Actual Salary"]
    -
    comparison["Predicted Salary"]
)

print("\nSample predictions:")

print(
    comparison.head(10)
)

# -----
# 20. FINAL MESSAGE
# -----

print("\n")
print("=" * 60)
print("SALARY PREDICTION COMPLETED SUCCESSFULLY!")
print("=" * 60)
```

ERROR: Salary Data.xlsx was not found!

Python searched here:
C:\Users\tvija\Downloads\Salary Data.xlsx

Excel files currently in Downloads:
Book2.xlsx
CRT Attendance_Above_70_Sectionwise.xlsx
HCL_Slot wise_Day Wise_Lab Wise - students list.xlsx
IV yr TUITION FEE DUE STUDENTS LIST AS ON 02-09-2026 (MODULE EXAM ON 03.09-2026) 2.xlsx
kidney_disease.xlsx
Vignan Foundation Eligible list .xlsx

In [35]: `print("results_df" in globals())`

False

In [38]: `print(df.columns.tolist())`

['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

In [41]: `import pandas as pd`

```
file_path = r"C:\Users\tvija\Downloads\salary_mlops\data\raw\salary_data.xlsx"
```

```
df = pd.read_excel(file_path)
```

```
print("Salary dataset loaded successfully!")
```

```
print("Shape:", df.shape)
```

```
print("\nColumns:")
```

```
print(df.columns.tolist())
```

```
print("\nFirst 5 rows:")
```

```
display(df.head())
```

Salary dataset loaded successfully!

Shape: (1000, 14)

Columns:
['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

First 5 rows:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	1052	2/3/2023	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob
1	1093	4/21/2023	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
2	1015	9/21/2023	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
3	1072	8/24/2023	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
4	1061	3/24/2023	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

In [45]: `import os`

```
model_folder = r"C:\Users\tvija\Downloads\salary_mlops\models"
```

```
os.makedirs(model_folder, exist_ok=True)
```

```
model_path = os.path.join(
```

```
    model_folder,
```

```
    "salary_model.pkl"
```

```
)
```

```
print("Model folder created successfully!")
print("Model path:", model_path)

Model folder created successfully!
Model path: C:\Users\tvija\Downloads\salary_mlops\models\salary_model.pkl
```

In [47]:

```
import os

processed_folder = r"C:\Users\tvija\Downloads\salary_mlops\data\processed"

os.makedirs(processed_folder, exist_ok=True)

processed_file = os.path.join(
    processed_folder,
    "processed_salary_data.xlsx"
)

print("Processed folder created successfully!")
print("Processed file:", processed_file)

Processed folder created successfully!
Processed file: C:\Users\tvija\Downloads\salary_mlops\data\processed\processed_salary_data.xlsx
```

In [51]:

```
# =====
# SALES MODEL TRAINING
# =====

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Target
X = df.drop("Sales_Amount", axis=1)
y = df["Sales_Amount"]

# Convert date
X["Sale_Date"] = pd.to_datetime(X["Sale_Date"], errors="coerce")

X["Sale_Year"] = X["Sale_Date"].dt.year
X["Sale_Month"] = X["Sale_Date"].dt.month
X["Sale_Day"] = X["Sale_Date"].dt.day

X.drop("Sale_Date", axis=1, inplace=True)

# Remove ID / combined column
X.drop(
    ["Product_ID", "Region_and_Sales_Rep"],
    axis=1,
    errors="ignore",
    inplace=True
)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Identify columns automatically
numeric_features = X.select_dtypes(
    include=np.number
).columns.tolist()

categorical_features = X.select_dtypes(
    include="object"
).columns.tolist()

# Preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            StandardScaler(),
            numeric_features
        ),
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        )
    ]
)

# Models
models = {
    "Linear Regression": LinearRegression(),

    "Decision Tree": DecisionTreeRegressor(
        random_state=42
    ),

    "Random Forest": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingRegressor(
        random_state=42
    )
}

# Train models
results = []
trained_models = {}

for name, model in models.items():
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)
```

```

y_pred = pipeline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

results.append({
    "Model": name,
    "MAE": mae,
    "MSE": mse,
    "RMSE": rmse,
    "R2": r2
})

trained_models[name] = pipeline

# Create results dataframe
results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by="R2",
    ascending=False
).reset_index(drop=True)

print("MODEL COMPARISON")
print("n" * 70)

print(results_df.to_string(index=False))

# Best model
best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

print("\nBest Model:", best_model_name)
print("Best R2 Score:", results_df.iloc[0]["R2"])

```

C:\Users\tvija\AppData\Local\Temp\ipykernel_9828\1375851328.py:58: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```

categorical_features = X.select_dtypes(
MODEL COMPARISON
=====
      Model      MAE      MSE      RMSE      R2
Linear Regression  2749.051353  9.612668e+06  3100.430343  -0.040183
Gradient Boosting  2746.153273  9.965138e+06  3156.760693  -0.078324
Random Forest     2775.099575  1.001417e+07  3164.518016  -0.083630
Decision Tree     3587.440050  1.779401e+07  4218.294801  -0.925483

```

```

Best Model: Linear Regression
Best R2 Score: -0.040183324832737455

```

```

In [ ]: import sys

!{sys.executable} -m pip install mlflow

```

```

In [54]: import mlflow
import mlflow.sklearn

print("MLflow version:", mlflow.__version__)

MLflow version: 3.16.0

```

```

In [55]: import mlflow
import mlflow.sklearn

mlflow.set_experiment(
    "Salary Data"
)

print("MLflow experiment ready!")

```

2026/09/09 19:23:16 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/09/09 19:23:16 INFO mlflow.store.db.utils: Updating database tables
2026/09/09 19:23:18 INFO mlflow.tracking.fluent: Experiment with name 'Salary Data' does not exist. Creating a new experiment.
MLflow experiment ready!

```

In [57]: import mlflow
import mlflow.sklearn
import numpy as np

for name, model in trained_models.items():
    with mlflow.start_run(
        run_name=name
    ):
        predictions = model.predict(
            X_test
        )

        mae = mean_absolute_error(
            y_test,
            predictions
        )

        mse = mean_squared_error(
            y_test,
            predictions
        )

        rmse = np.sqrt(mse)

        r2 = r2_score(
            y_test,
            predictions
        )

        mlflow.set_tag(
            "model_name",
            name
        )

        mlflow.log_metric(
            "MAE",
            mae
        )

        mlflow.log_metric(
            "MSE",
            mse
        )

        mlflow.log_metric(
            "RMSE",
            rmse
        )

        mlflow.log_metric(
            "R2",
            r2
        )

```

```

)

mlflow.sklearn.log_model(
    sk_model=model,
    name="model",
    input_example=X_test.head(1)
)

print(
    name,
    "logged successfully!"
)

```

C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\mlflow\types\utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values' <<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>>_ for more details.

```
warnings.warn(
Downloading artifacts: 0%|          | 0/7 [00:00<, ?it/s]
Linear Regression logged successfully!
```

C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\mlflow\types\utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values' <<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>>_ for more details.

```
warnings.warn(
Downloading artifacts: 0%|          | 0/7 [00:00<, ?it/s]
Decision Tree logged successfully!
```

C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\mlflow\types\utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values' <<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>>_ for more details.

```
warnings.warn(
Downloading artifacts: 0%|          | 0/7 [00:00<, ?it/s]
Random Forest logged successfully!
```

C:\Users\tvija\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\mlflow\types\utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values' <<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>>_ for more details.

```
warnings.warn(
Downloading artifacts: 0%|          | 0/7 [00:00<, ?it/s]
Gradient Boosting logged successfully!
```

In [58]: `|git --version`

```
git version 2.49.0.windows.1
```

In [59]: `|import os`

```
print(
    os.path.expanduser("~")
)
```

C:\Users\tvija

In [61]: `|import os`

```
project_folder = r"C:\Users\tvija\Downloads\salary_prediction_mlops"

os.makedirs(
    project_folder,
    exist_ok=True
)

print("Project folder created successfully!")
print(project_folder)
```

Project folder created successfully!

C:\Users\tvija\Downloads\salary_prediction_mlops

In []: `|import shutil
|import os`

```
source_file = r"C:\Users\user\Downloads\Salary Data.xlsx"

destination_file = os.path.join(
    project_folder,
    "Salary Data.xlsx"
)

shutil.copy(
    source_file,
    destination_file
)

print("Salary Data.xlsx copied successfully!")
print(destination_file)
```

In [62]: `|%cd "C:\Users\user\Downloads\salary_prediction_mlops"`

```
[WinError 3] The system cannot find the path specified: 'C:\\Users\\user\\Downloads\\salary_prediction_mlops'
C:\Users\tvija
```

In [63]: `|import os`

```
print(
    os.listdir()
)
```

```
['.android', '.cache', '.copilot', '.eclipse', '.emulator_console_auth.token', '.gitconfig', '.gradle', '.ipynb_checkpoints', '.ipython', '.jupyter', '.local', '.p2', '.vscode', '.vscode-shared', 'aks
hitha.pdf', 'AndroidStudioProjects', 'AppData', 'Application Data', 'best_model.joblib', 'best_model.pkl', 'cleaned_dataset.csv', 'cleaned_heart.csv', 'Contacts', 'Cookies', 'Documents', 'Downloads',
'eclipse', 'eclipse-workspace', 'ecommerce_project', 'exp2_231fa04637.ipynb', 'exp_4_231fa04637.ipynb', 'Favorites', 'heart.csv', 'Links', 'Local Settings', 'mlflow.db', 'mlops_231fa04637(ex1).ipyn
b', 'mlruns', 'Music', 'My Documents', 'NetHood', 'notebook', 'NTUSER.DAT', 'ntusen.dat.LOG1', 'ntusen.dat.LOG2', 'NTUSER.DAT(a2332f17-cdbf-11ec-8680-002248483d79).TxR.0.regtrans-ms', 'NTUSER.DAT(a233
2f17-cdbf-11ec-8680-002248483d79).TxR.1.regtrans-ms', 'NTUSER.DAT(a2332f17-cdbf-11ec-8680-002248483d79).TxR.2.regtrans-ms', 'NTUSER.DAT(a2332f17-cdbf-11ec-8680-002248483d79).TxR.bif', 'NTUSER.DAT(a233
2f18-cdbf-11ec-8680-002248483d79).TM.bif', 'NTUSER.DAT(a2332f18-cdbf-11ec-8680-002248483d79).TMContainer00000000000000000001.regtrans-ms', 'NTUSER.DAT(a2332f18-cdbf-11ec-8680-002248483d79).TMContainer
000000000000000002.regtrans-ms', 'ntusen.ini', 'OneDrive', 'ppt.pptx', 'PrintHood', 'Recent', 'Sales_project', 'Saved Games', 'Searches', 'SendTo', 'Start Menu', 'Templates', 'Untitled.ipynb', 'Unti
tled1.ipynb', 'Untitled2.ipynb', 'Untitled3.ipynb', 'Videos']
```

In [64]: `|git init`

```
Initialized empty Git repository in C:/Users/tvija/.git/
```

In [68]: `|rmdir /s /q .git`

In [71]: `|import os`

```
os.chdir(r"C:\Users\tvija\Downloads\salary_prediction_mlops")

print(os.getcwd())
```

C:\Users\tvija\Downloads\salary_prediction_mlops

In [72]: `|git init`

```
Initialized empty Git repository in C:/Users/tvija/Downloads/salary_prediction_mlops/.git/
```

In [73]: `|with open(".gitignore", "w") as f:`

```
    f.write("""# Gradle
.gradle/
build/

# IDE
.idea/
.vscode/

# Python
__pycache_/
```

```
.ipynb_checkpoints/  
  
# MLflow  
mlruns/  
mlflow.db  
  
# Model files  
*.pkl  
*.joblib  
""")  
  
print(".gitignore created!")  
  
.gitignore created!
```

In [74]:

```
git add .
```

In [76]:

```
import os  
print(os.getcwd())  
print(os.listdir())  
  
C:\Users\tvija\Downloads\salary_prediction_mlops  
['.git', '.gitignore']
```

In [9]:

```
git config --global user.email "231fa04564@gmail.com"  
git config --global user.name "Muvva Veda Sri"
```

In [10]:

```
git config --global user.email  
git config --global user.name  
  
231fa04564@gmail.com  
Muvva Veda Sri
```

In []: